

# Informe PDyTR

**Materia: Programación Distribuida y Tiempo Real**

**Práctica 2**

**Alumno: Taiel Alen Nunes**

1. Buscar y utilizar en vagrant una versión más reciente de Ubuntu LTS que la dada en la explicación de práctica. Identificar versión, origen y si tuviera diferencias, explicar brevemente. Mostrar con una captura de pantalla la identificación en VirtualBox la máquina virtual y el inicio de la terminal en funcionamiento. Consulte a un asistente de IA (deje constancia de cuál) si hay alguna versión en particular que puede ser mejor que otra y por qué.

## 1.1 - ¿Que version utiliza el vagantfile de la catedra?

- El vagantfile de la catedra presenta la box `bento/ubuntu-22.04` utilizada para entornos **Ubuntu 22.04**.

```
config.vm.define "vm#{id}" do |machine|  
  ...  
  machine.vm.box = "bento/ubuntu-22.04"  
  ...  
end
```

## 1.2 - Consulta a la IA - DeepSeek

- ¿Cuál es la versión más reciente y más recomendada?

## Versión más reciente de Ubuntu LTS en Vagrant

### 📌 La versión LTS más reciente y su soporte

Actualmente, la versión LTS (Long Term Support) más reciente de Ubuntu es **Ubuntu 24.04 LTS (Noble Numbat)**, lanzada en abril de 2024.

A continuación se presenta una comparativa del ciclo de vida y soporte de las últimas versiones LTS, dato clave para elegir la más conveniente:

| Versión de Ubuntu | Nombre en clave | Soporte Estándar | Soporte Extendido (ESM)     |
|-------------------|-----------------|------------------|-----------------------------|
| Ubuntu 24.04 LTS  | Noble Numbat    | Mayo 2029        | Abril 2034 (con Ubuntu Pro) |
| Ubuntu 22.04 LTS  | Jammy Jellyfish | Junio 2027       | Abril 2032 (con Ubuntu Pro) |
| Ubuntu 20.04 LTS  | Focal Fossa     | Mayo 2025        | Abril 2030 (con Ubuntu Pro) |

**Ubuntu 24.04 LTS** es la opción más moderna y con el soporte más prolongado, lo que la convierte en la elección ideal para proyectos a largo plazo.

- ¿Qué box de Ubuntu 24.04 utilizar?

### ⚠ El "problema" de las boxes oficiales en Ubuntu 24.04

Es importante tener en cuenta que **Canonical (la empresa detrás de Ubuntu)** ha dejado de publicar **boxes oficiales de Vagrant a partir de la versión 24.04 LTS**. Esto se debe a un cambio en la licencia del software Vagrant.

Por lo tanto, **NO existe una box** `ubuntu/noble64` como sí existen para versiones anteriores (por ejemplo, `ubuntu/focal64` para 20.04 o `ubuntu/jammy64` para 22.04). Esto significa que, para usar Ubuntu 24.04 en Vagrant, es necesario recurrir a boxes mantenidas por terceros.

Además, se han reportado múltiples problemas de compatibilidad con la box `bento/ubuntu-22.04` en entornos VirtualBox, que van desde timeouts en el arranque hasta errores que indican que no soporta el proveedor de virtualización. Por esta razón, se recomienda evitar su uso.

## 💡 ¿Qué box usar para Ubuntu 24.04?

Dado que no hay boxes oficiales de Canonical para 24.04, se recomienda usar una de las siguientes alternativas, que son más estables que `bento` :

1. `cloud-image/ubuntu-24.04` : Esta box es publicada por HashiCorp, la empresa creadora de Vagrant. Aunque no es "oficial" de Canonical, es la opción más recomendada y estandarizada para Ubuntu 24.04 en Vagrant Cloud.
2. `alvistack/ubuntu-24.04` : Esta es otra box mantenida por terceros que ha recibido buenas valoraciones recientes y se actualiza con frecuencia. Es una alternativa sólida si la de HashiCorp presentara inconvenientes.

- Investigando mas a fondo sobre las recomendaciones de DeepSeek podemos ver que la versión más actualizada es la de `cloud-image` cuya ultima release fue hace 10 dias (a dia de la fecha "22-09-2026").
- <https://portal.cloud.hashicorp.com/vagrant/discover>

| Box name                                                                                                                  | Latest Version | Downloads | Latest Release | Providers                                                      | Architectures                         |
|---------------------------------------------------------------------------------------------------------------------------|----------------|-----------|----------------|----------------------------------------------------------------|---------------------------------------|
|  <code>cloud-image/ubuntu-24.04</code> | 20260911.0.0   | 18,197    | 10 days ago    | <code>qemu</code> <code>libvirt</code> <code>virtualbox</code> | <code>amd64</code> <code>arm64</code> |

| Box name                                                                                                                | Latest Version | Downloads | Latest Release | Providers                                    | Architectures      |
|-------------------------------------------------------------------------------------------------------------------------|----------------|-----------|----------------|----------------------------------------------|--------------------|
|  <code>alvistack/ubuntu-24.04</code> | 20260108.1.1   | 11,158    | 256 days ago   | <code>virtualbox</code> <code>libvirt</code> | <code>amd64</code> |

- En principio descargue y probé la de `cloud-image` pero presentó problemas en el arranque de las VMs.
- Por lo tanto, decidí probar la segunda opcion recomendada por DeepSeek `alvistack/ubuntu-24.04` . Esta ultima funcionó sin inconvenientes.

### 1.3 - Diferencias

- Revisando la Overview de `alvistack/ubuntu-24.04` :
  - <https://portal.cloud.hashicorp.com/vagrant/discover/alvistack/ubuntu-24.04>
  - La box `alvistack/ubuntu-24.04` sigue la misma estructura estandar establecida por Vagrant, por lo que está preparada para funcionar con los mismos parámetros que ya están configurados en el Vagrantfile proporcionado por la catedra.

- Las **única diferencia** es que esta es para la **versión 24.04**.

#### 1.4 - ¿Cómo aplicar la nueva box al Vagrantfile?

- Para cambiar el `Vagrantfile` y usar la box recomendada, tan solo debemos modificar la línea `vm.box` en la configuración del VagrantFile:

```
Vagrant.configure("2") do |config|

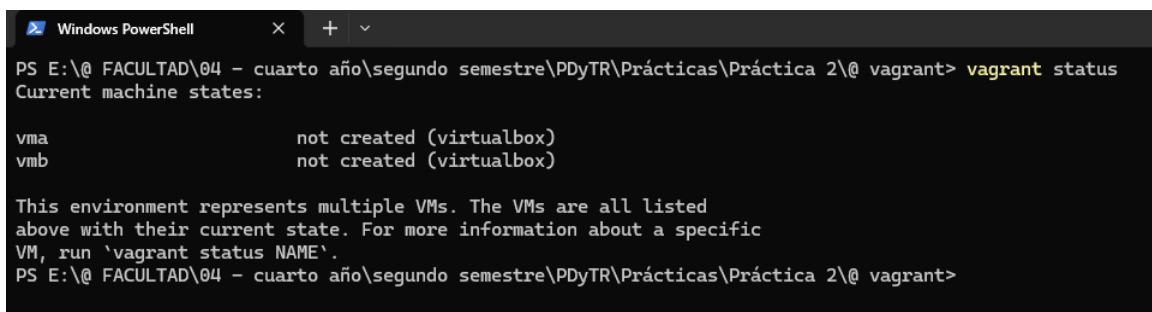
  ("a".."b").each do |id|
    config.vm.define "vm#{id}" do |machine|
      machine.vm.box = "alvistack/ubuntu-24.04" # nueva box mas copada
      machine.vm.network "private_network", ip: "192.168.56.1#{id == 'a' ? '0' : '1'}"
      machine.vm.hostname = "vm#{id}"
      machine.vm.synced_folder "..", "/home/vagrant/tp2"
    end
  end
end
```

- Una vez configurado el `VagrantFile` con la nueva box, podemos proceder levantando las máquinas virtuales.

#### 1.5 - Creación de las VMS

##### 1. Verificar estado de las VMs

```
vagrant status
```



```
Windows PowerShell
PS E:\@ FACULTAD\04 - cuarto año\segundo semestre\PDyTR\Prácticas\Práctica 2\@ vagrant> vagrant status
Current machine states:

vma                        not created (virtualbox)
vmb                        not created (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run 'vagrant status NAME'.
PS E:\@ FACULTAD\04 - cuarto año\segundo semestre\PDyTR\Prácticas\Práctica 2\@ vagrant>
```

- podemos ver que las VMs no se encuentran creadas, por lo que no hace falta ejecutar el `vagrant destroy`.

##### 2. Crear y Levantar las VMs

```
vagrant up
```

```

PS E:\@ FACULTAD\04 - cuarto año\segundo semestre\PDyTR\@ Prácticas\Práctica 2\@ vagrant> vagrant up
Bringing machine 'vma' up with 'virtualbox' provider...
Bringing machine 'vmb' up with 'virtualbox' provider...
==> vma: Box 'alvistack/ubuntu-24.04' could not be found. Attempting to find and install...
vma: Box Provider: virtualbox
vma: Box Version: >= 0
==> vma: Loading metadata for box 'alvistack/ubuntu-24.04'
vma: URL: https://vagrantcloud.com/api/v2/vagrant/alvistack/ubuntu-24.04
==> vma: Adding box 'alvistack/ubuntu-24.04' (v20260108.1.1) for provider: virtualbox (amd64)
vma: Downloading: https://vagrantcloud.com/alvistack/boxes/ubuntu-24.04/versions/20260108.1.1/providers/virtualbox/amd64/vagrant.box
vma:
==> vma: Successfully added box 'alvistack/ubuntu-24.04' (v20260108.1.1) for 'virtualbox (amd64)'!
==> vma: Importing base box 'alvistack/ubuntu-24.04'...
==> vma: Matching MAC address for NAT networking...
==> vma: Checking if box 'alvistack/ubuntu-24.04' version '20260108.1.1' is up to date...
==> vma: Setting the name of the VM: vma
==> vma: Clearing any previously set network interfaces...
==> vma: Preparing network interfaces based on configuration...
vma: Adapter 1: nat
vma: Adapter 2: hostonly
==> vma: Forwarding ports...
vma: 22 (guest) => 2222 (host) (adapter 1)
==> vma: Running 'pre-boot' VM customizations...
==> vma: Booting VM...
==> vma: Waiting for machine to boot. This may take a few minutes...
vma: SSH address: 127.0.0.1:2222
vma: SSH username: vagrant
vma: SSH auth method: private key
vma:
vma: Vagrant insecure key detected. Vagrant will automatically replace
vma: this with a newly generated keypair for better security.
vma:
vma: Inserting generated public key within guest...
vma: Removing insecure key from the guest if it's present...
vma: Key inserted! Disconnecting and reconnecting using new SSH key...
==> vma: Machine booted and ready!

```

```

==> vmb: Box 'alvistack/ubuntu-24.04' could not be found. Attempting to find and install...
vmb: Box Provider: virtualbox
vmb: Box Version: >= 0
==> vmb: Loading metadata for box 'alvistack/ubuntu-24.04'
vmb: URL: https://vagrantcloud.com/api/v2/vagrant/alvistack/ubuntu-24.04
==> vmb: Adding box 'alvistack/ubuntu-24.04' (v20260108.1.1) for provider: virtualbox (amd64)
vmb: Downloading: https://vagrantcloud.com/alvistack/boxes/ubuntu-24.04/versions/20260108.1.1/providers/virtualbox/amd64/vagrant.box
vmb:
==> vmb: Successfully added box 'alvistack/ubuntu-24.04' (v20260108.1.1) for 'virtualbox (amd64)'!
==> vmb: Importing base box 'alvistack/ubuntu-24.04'...
==> vmb: Matching MAC address for NAT networking...
==> vmb: Checking if box 'alvistack/ubuntu-24.04' version '20260108.1.1' is up to date...
==> vmb: Setting the name of the VM: vmb
==> vmb: Fixed port collision for 22 => 2222. Now on port 2200.
==> vmb: Clearing any previously set network interfaces...
==> vmb: Preparing network interfaces based on configuration...
vmb: Adapter 1: nat
vmb: Adapter 2: hostonly
==> vmb: Forwarding ports...
vmb: 22 (guest) => 2200 (host) (adapter 1)
==> vmb: Running 'pre-boot' VM customizations...
==> vmb: Booting VM...
==> vmb: Waiting for machine to boot. This may take a few minutes...
vmb: SSH address: 127.0.0.1:2200
vmb: SSH username: vagrant
vmb: SSH auth method: private key
vmb:
vmb: Vagrant insecure key detected. Vagrant will automatically replace
vmb: this with a newly generated keypair for better security.
vmb:
vmb: Inserting generated public key within guest...
vmb: Removing insecure key from the guest if it's present...
vmb: Key inserted! Disconnecting and reconnecting using new SSH key...
==> vmb: Machine booted and ready!

```

- Esperamos unos minutos a que se creen las VMs `vma` y `vmb`

### 3. Verificar si que estén Corriendo en VirtualBox

vagrant status

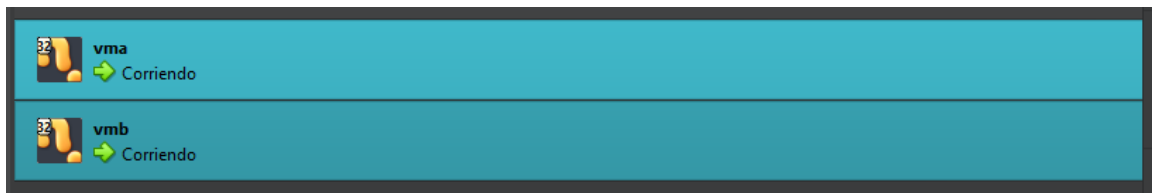
```

PS E:\@ FACULTAD\04 - cuarto año\segundo semestre\PDyTR\@ Prácticas\Práctica 2\@ vagrant> vagrant status
Current machine states:

vma                      running (virtualbox)
vmb                      running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run `vagrant status NAME`.

```



- Ya tenemos la `vma` y `vmb` funcionando.

#### 4. Entrar a las VMs desde terminal

```
vagrant ssh vma
```

```
PS E:\@ FACULTAD\04 - cuarto año\segundo semestre\PDyTR\@ Prácticas\Práctica 2\@ vagrant> vagrant ssh vma
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 22 09:04:27 PM UTC 2026

System load:          0.16
Usage of /:            4.0% of 109.68GB
Memory usage:         44%
Swap usage:           0%
Processes:            116
Users logged in:      0
IPv4 address for eth0: 10.0.2.15
IPv6 address for eth0: fd17:625c:f037:2:a00:27ff:fe88:8e11

Expanded Security Maintenance for Applications is not enabled.

375 updates can be applied immediately.
218 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

5 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

vagrant@vma:~$
```

```
vagrant ssh vmb
```

```

PS E:\@ FACULTAD\04 - cuarto año\segundo semestre\PDyTR\@ Prácticas\Práctica 2\@ vagrant> vagrant ssh vmb
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 22 09:04:31 PM UTC 2026

System load:          0.0
Usage of /:           4.0% of 109.68GB
Memory usage:         44%
Swap usage:           0%
Processes:            114
Users logged in:      0
IPv4 address for eth0: 10.0.2.15
IPv6 address for eth0: fd17:625c:f037:2:a00:27ff:fe88:8e11

Expanded Security Maintenance for Applications is not enabled.

375 updates can be applied immediately.
218 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

5 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

vagrant@vmb:~$ |

```

- En este punto ya tenemos las VMs listas para realizar los experimentos.

## 1.5 - Conclusión:

- La box utilizada por en Vagrantfile de la catedrá es `bento/ubuntu-22.04` utilizada para **Ubuntu 22.04**.
- La box que se usará para los experimentos es `alvistack/ubuntu-24.04` utilizada para **Ubuntu 24.04** (la versión más actual),
- Esta es una box **no oficial mantenida por terceros**, ya que Canonical dejó de publicar boxes para esta versión de Ubuntu.
- La única diferencia es la version más actual **Ubuntu 24.04**.

2

Ejecutar los experimentos del ejercicio 4 a), b) y c) de la práctica anterior en dos computadoras físicamente diferentes. Se copia aquí el enunciado para mayor comodidad: comunicar  $10^1$ ,  $10^2$ ,  $10^3$ ,  $10^4$ ,  $10^5$  y  $10^6$  bytes desarrollando experimentos que muestre el tiempo que toma en el cliente

a) La función `write(...)` para cada cantidad de datos. Tomando C como ejemplo:

```

t0 =
write(...)

```

```
t1 = ...  
tiempo = t1 - t0;
```

b) La función `read(...)`. Tomando C como ejemplo:

```
t0 = ...  
n = read(sockfd, buffer, 255);  
t1 = ...  
tiempo = t1 - t0;
```

c) Grafique y explique los resultados obtenidos. En particular, identifique si las diferencias de tiempos son proporcionales a las cantidades de datos, por ej: ¿`write(...)` con 1000 bytes toma 10 veces más tiempo que con 100 bytes? Identifique si el tiempo de la función `read()` se mantiene constante, dado que involucra siempre la misma cantidad de datos.

Proveer una descripción mínima de las computadoras usadas (CPU, RAM, OS). Aclare si las computadoras están en la misma red local o en dos redes locales diferentes (en este último caso, es posible que deban modificar al menos un router si ambas computadoras tienen acceso vía NAT). Pueden utilizar las computadoras del aula o sus propias computadoras. Documentar y entregar no solamente el código de los procesos de comunicaciones sino también el propio experimento.

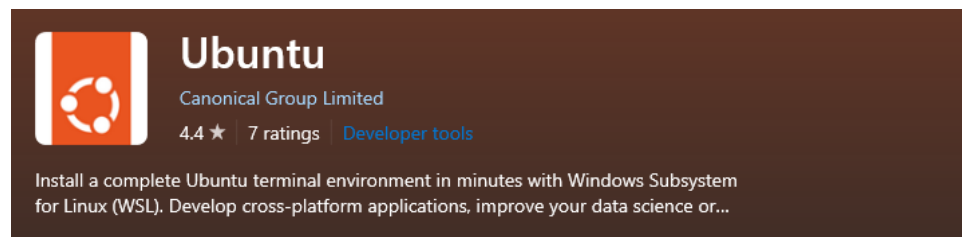
## **2.1 - Descripción de las computadoras y de la red**

- Ambas computadoras utilizan **Windows** como **sistema operativo anfitrión** y ejecutan los programas dentro de distribuciones Linux mediante **WSL (*Windows Subsystem for Linux*)**.
- Las computadoras pertenecen a la **misma red local**, aunque utilizan diferentes medios de conexión: la computadora servidor está conectada al router mediante un cable Ethernet, mientras que la computadora cliente se conecta mediante un adaptador Wi-Fi.
- Por lo tanto, la comunicación incluye un tramo inalámbrico, cuya variabilidad puede influir en los tiempos medidos.
- **Computadora 1 - Servidor:**

- Ejecuta el programa `server_timing` dentro de Ubuntu mediante WSL 2. Sus características de hardware se muestran en la siguiente captura.
- Información básica del **Hardware**:

| Device Specifications |                                                                                                                                                                  | Copy | ^ |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---|
| Device Name           | Taiel-PC                                                                                                                                                         |      |   |
| Processor             | Intel(R) Core(TM) i5-8400 CPU @ 2.80GHz 2.81 GHz                                                                                                                 |      |   |
| Installed RAM         | 16.0 GB                                                                                                                                                          |      |   |
| Graphics Card         | NVIDIA GeForce RTX 2060 SUPER (8 GB)                                                                                                                             |      |   |
| Storage               | 932 GB SSD WD Green 2.5 1000GB, 477 GB SSD XPG SPECTRIX S40G, 932 GB SSD WD Green 2.5 1000GB, 1.82 TB HDD WDC WD20PURX-64PFUY0, 1.82 TB HDD WDC WD20EZRZ-00Z5HB0 |      |   |

- Utilizará la **terminal de Ubuntu**.

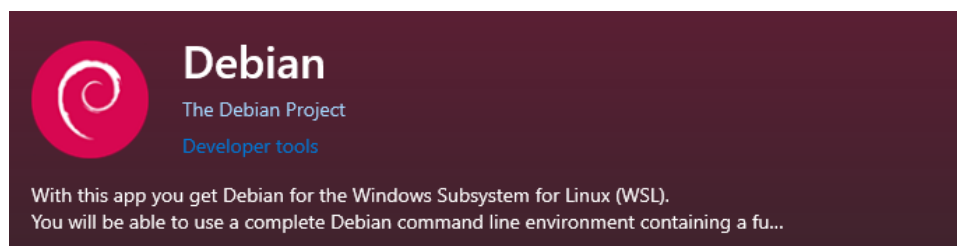


## • Computadora 2 - Cliente:

- Ejecuta el programa `client_timing_chart` dentro de Debian mediante WSL. En esta computadora se realizan las mediciones de tiempo y se generan los archivos de resultados.
- Información básica del **Hardware**:

|                        |                                                                 |
|------------------------|-----------------------------------------------------------------|
| Nombre del dispositivo | DESKTOP-3UBP3KG                                                 |
| Procesador             | Intel(R) Pentium(R) Gold G6405 CPU @ 4.10GHz 4.10 GHz           |
| RAM instalada          | 8,00 GB (7,82 GB utilizable)                                    |
| Tarjeta gráfica        | Intel(R) UHD Graphics 610 (128 MB)                              |
| Almacenamiento         | 238 GB SSD XPG SPECTRIX S40G, 466 GB HDD WDC WD5000AAKX-00ERMA0 |

- Utilizará la **terminal de Debian**.



## 2.2 - Preparación del entorno de comunicación

Para realizar el experimento, se preparó el acceso desde la computadora cliente, que ejecuta Debian, al proceso servidor, alojado en Ubuntu dentro de WSL 2.

En la configuración utilizada, Windows y Ubuntu tienen direcciones IP diferentes. El cliente puede acceder a Windows mediante la red local, pero no directamente a la red interna de WSL. Por este motivo, se configuró un **proxy TCP** que recibe las conexiones en Windows y las comunica con el servidor en Ubuntu. Además, se agregó una regla de firewall para permitir dichas conexiones.

### **¿Qué direcciones IP se necesitan?**

- Se identificaron dos direcciones en la computadora servidor.
- En **PowerShell de Windows**, se ejecutó:

```
ipconfig
```

La IPv4 del adaptador Ethernet conectado al router fue **192.168.100.6**. Esta dirección será utilizada por el cliente como destino.

```

Windows PowerShell
PS C:\Users\taiel> ipconfig

Windows IP Configuration

Ethernet adapter Radmin VPN:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : fdfd::1a32:a916
    Link-local IPv6 Address . . . . . : fe80::af4c:a293:fcc0:71c7%13
    IPv4 Address. . . . . : 26.50.169.22
    Subnet Mask . . . . . : 255.0.0.0
    Default Gateway . . . . . : 26.0.0.1

Ethernet adapter vEthernet (Default Switch):

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::a2da:ae12:f8f8:eeeb%26
    IPv4 Address. . . . . : 172.31.112.1
    Subnet Mask . . . . . : 255.255.240.0
    Default Gateway . . . . . : 

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : 2803:9800:90ab:8b7e:2452:9745:946e:4d49
    Temporary IPv6 Address. . . . . : 2803:9800:90ab:8b7e:80d6:a796:3c35:bb60
    Link-local IPv6 Address . . . . . : fe80::a761:1013:8a5c:4312%7
    IPv4 Address. . . . . : 192.168.100.6
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::1%7
                             192.168.100.1

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::ae:2e0b:9ad3:b99a%4
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Tunnel adapter Teredo Tunneling Pseudo-Interface:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : 2001:0:14c9:cd04:64:81ee:4561:8743
    Link-local IPv6 Address . . . . . : fe80::64:81ee:4561:8743%14
    Default Gateway . . . . . : 

Ethernet adapter vEthernet (WSL):

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::18d5:e4ae:21b9:42bc%43
    IPv4 Address. . . . . : 172.31.16.1
    Subnet Mask . . . . . : 255.255.240.0
    Default Gateway . . . . . : 

```

- Luego, en la **terminal de Ubuntu**, se ejecutó:

```
hostname -I
```

La dirección interna de Ubuntu fue **172.31.25.103** . Esta dirección será utilizada por el proxy para conectarse con el proceso servidor.

```

Select taiel@Taiel-PC: /mnt/e/@ FACULTAD/04 - cuarto año/segundo semestre/PDyTR/Prácticas/Práctica 2/02
taiel@Taiel-PC: /mnt/e/@ FACULTAD/04 - cuarto año/segundo semestre/PDyTR/Prácticas/Práctica 2/02$ hostname -I
172.31.25.103

```

| Dirección     | Entorno              | Uso                           |
|---------------|----------------------|-------------------------------|
| 192.168.100.6 | Windows del servidor | Punto de entrada desde la LAN |

| Dirección     | Entorno              | Uso                       |
|---------------|----------------------|---------------------------|
| 172.31.25.103 | Ubuntu dentro de WSL | Destino interno del proxy |

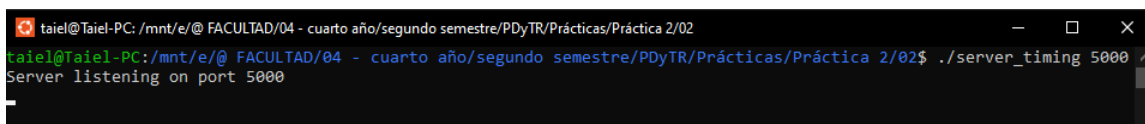
### ¿Cómo se inicia el servidor?

- Desde Ubuntu, en la carpeta del programa, se ejecutó:

```
./server_timing 5000
```

- El argumento `5000` indica el puerto TCP donde el servidor espera conexiones. El inicio se confirmó mediante el mensaje:

```
Server listening on port 5000
```



La terminal se mantuvo abierta durante el experimento. El servidor se inició antes que el cliente para que estuviera disponible al comenzar las pruebas.

### ¿Cómo se permite el acceso al servidor dentro de WSL?

- WSL 2 utiliza, en este caso, una red virtual con NAT. Por ello, iniciar el programa en Ubuntu no publica automáticamente su puerto en la interfaz de Windows accesible desde la LAN. Se utilizó `portproxy` para establecer ese acceso, siguiendo el procedimiento documentado por Microsoft.
- En **PowerShell del servidor, como administrador**, se ejecutó:

```
netsh interface portproxy add v4tov4 listenaddress=192.168.100.6 listenport=5000 connectaddress=172.31.25.103 connectport=5000
```

El comando **agrega un proxy** entre dos extremos IPv4 (`v4tov4`):

- `listenaddress` y `listenport`: indican dónde Windows recibe la conexión del cliente: `192.168.100.6:5000`.
- `connectaddress` y `connectport`: indican dónde el proxy se conecta con el servidor: `172.31.25.103:5000`.

El proxy establece la conexión hacia Ubuntu y transmite los datos en ambos sentidos.

- Además, se ejecutó:

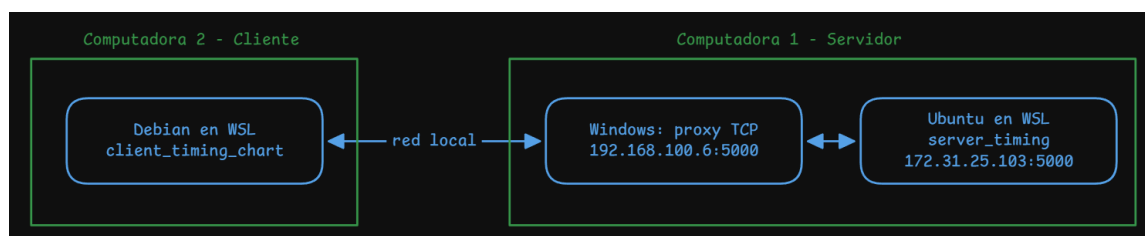
```
New-NetFirewallRule -DisplayName "PDyTR TCP 5000" -Direction Inbound -Action Allow  
-Protocol TCP -LocalPort 5000 -RemoteAddress LocalSubnet
```

Este comando creó una **regla de firewall**.

Esta regla, denominada **"PDyTR TCP 5000"**, permite tráfico entrante ( **Inbound** ) de tipo TCP dirigido al puerto local 5000, desde las subredes locales ( **LocalSubnet** ). Mientras el proxy proporciona el enlace hacia Ubuntu, el firewall permite acceder a su punto de entrada.

```
PS C:\Windows\system32> netsh interface portproxy add v4tov4 listenaddress= 192.168.100.6 listenport=5000 connectaddress=172.31.25.103 connectport=5000  
PS C:\Windows\system32> New-NetFirewallRule -DisplayName "PDyTR TCP 5000" -Direction Inbound -Action Allow -Protocol TCP -LocalPort 5000 -RemoteAddress LocalSubnet  
  
Name : {d1a242de-d0fc-b119-32c8f4440cd7}  
DisplayName : PDyTR TCP 5000  
Description :  
DisplayGroup :  
Group :  
Enabled : True  
Profile : Any  
Platform : {}  
Direction : Inbound  
Action : Allow  
EdgeTraversalPolicy : Block  
LooseSourceMapping : False  
LocalOnlyMapping : False  
Owner :  
PrimaryStatus : OK  
Status : The rule was parsed successfully from the store. (65536)  
EnforcementStatus : NotApplicable  
PolicyStoreSource : PersistentStore  
PolicyStoreSourceType : Local  
RemoteDynamicKeywordAddresses :  
PolicyAppId :
```

- La comunicación queda organizada de la siguiente manera. Las flechas representan el intercambio en ambos sentidos:



No fue necesario modificar el router porque ambas computadoras pertenecen a la misma LAN. El proxy se configuró en Windows para acceder a la red interna de WSL.

## ¿Cómo se comprueba la conexión antes del experimento?

- Desde PowerShell de la computadora cliente se ejecutó:

```
Test-NetConnection 192.168.100.6 -Port 5000
```

Windows PowerShell

```
PS C:\Users\mimat> Test-NetConnection 192.168.100.6 -Port 5000

ComputerName      : 192.168.100.6
RemoteAddress     : 192.168.100.6
RemotePort        : 5000
InterfaceAlias    : Wi-Fi 2
SourceAddress     : 192.168.100.10
TcpTestSucceeded  : True
```

- El resultado **TcpTestSucceeded : True** confirmó que se podía establecer una conexión TCP al punto de entrada configurado. Esta prueba verifica la conectividad; el intercambio de mensajes y las mediciones se realizan posteriormente con el programa cliente.
- Con esta preparación, el cliente debe utilizar **192.168.100.6** y el puerto **5000** como destino. El acceso a Ubuntu queda a cargo del proxy.
- Hay que coinsiderar que tanto este intermediario como WSL forman parte del entorno del experimento y **pueden influir en los tiempos obtenidos**.

### 2.3 - Ejecución y almacenamiento de resultados

- Con el servidor en ejecución, se inició el programa cliente desde la terminal de Debian de la computadora 2, indicando la dirección IP de Windows del servidor y el puerto configurado:

```
./client_timing_chart 192.168.100.6 5000
```

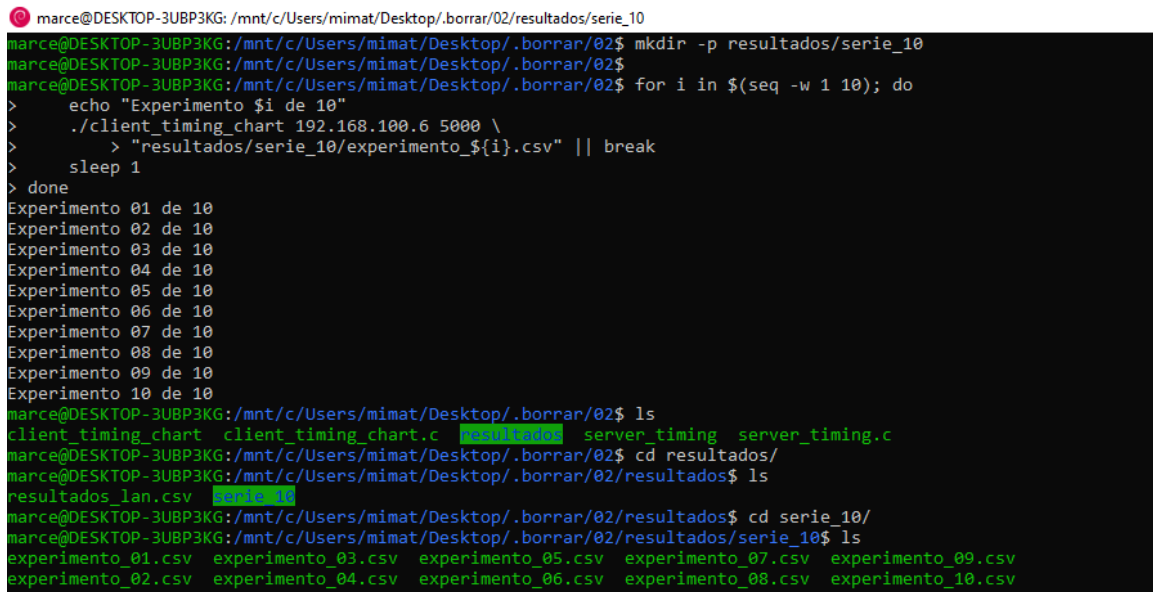
Se realizó una primera ejecución que permitió comprobar el intercambio completo y visualizar los resultados en la terminal.

```
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/02
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$ ./client_timing_chart 192.168.100.6 5000
bytes,repeticiones,write_avg_ns,read_avg_ns
10,10,3660.00,57539999.80
100,10,5760.00,57368839.80
1000,10,6680.00,57656899.80
10000,10,96470.00,40063769.90
100000,10,1049420.00,129308639.70
1000000,10,146814458.10,792025970.90
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$
```

- Luego, se realizaron diez ejecuciones completas, almacenando los resultados de cada una en un archivo CSV independiente:

```
mkdir -p resultados/serie_10

for i in $(seq -w 1 10); do
    echo "Experimento $i de 10"
    ./client_timing_chart 192.168.100.6 5000 \
        > "resultados/serie_10/experimento_${i}.csv" ||
break
    sleep 1
done
```



```
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/02/resultados/serie_10
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$ mkdir -p resultados/serie_10
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$ for i in $(seq -w 1 10); do
>     echo "Experimento $i de 10"
>     ./client_timing_chart 192.168.100.6 5000 \
>         > "resultados/serie_10/experimento_${i}.csv" || break
>     sleep 1
> done
Experimento 01 de 10
Experimento 02 de 10
Experimento 03 de 10
Experimento 04 de 10
Experimento 05 de 10
Experimento 06 de 10
Experimento 07 de 10
Experimento 08 de 10
Experimento 09 de 10
Experimento 10 de 10
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$ ls
client_timing_chart  client_timing_chart.c  resultados  server_timing  server_timing.c
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02$ cd resultados/
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02/resultados$ ls
resultados_lan.csv  serie_10
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02/resultados$ cd serie_10/
marce@DESKTOP-3UBP3KG:/mnt/c/Users/mimat/Desktop/.borrar/02/resultados/serie_10$ ls
experimento_01.csv  experimento_03.csv  experimento_05.csv  experimento_07.csv  experimento_09.csv
experimento_02.csv  experimento_04.csv  experimento_06.csv  experimento_08.csv  experimento_10.csv
```

El bucle ejecuta el cliente diez veces y el operador `>` guarda su salida en archivos numerados desde `experimento_01.csv` hasta `experimento_10.csv`. Entre ejecuciones se introduce una pausa de un segundo, fuera de los intervalos medidos.

Cada CSV contiene el tamaño del mensaje, la cantidad de repeticiones y los tiempos promedio de escritura y lectura en nanosegundos:

```
bytes,repeticiones,write_avg_ns,read_avg_ns
```

- Los archivos se conservaron para calcular los promedios generales y elaborar las tablas y los gráficos del análisis.

| Name                                                                                                 | Date modified   | Type                 | Size |
|------------------------------------------------------------------------------------------------------|-----------------|----------------------|------|
|  experimento_01.csv | 3/10/2026 23:31 | Microsoft Excel C... | 1 KB |
|  experimento_02.csv | 3/10/2026 23:31 | Microsoft Excel C... | 1 KB |
|  experimento_03.csv | 3/10/2026 23:31 | Microsoft Excel C... | 1 KB |
|  experimento_04.csv | 3/10/2026 23:32 | Microsoft Excel C... | 1 KB |
|  experimento_05.csv | 3/10/2026 23:32 | Microsoft Excel C... | 1 KB |
|  experimento_06.csv | 3/10/2026 23:32 | Microsoft Excel C... | 1 KB |
|  experimento_07.csv | 3/10/2026 23:32 | Microsoft Excel C... | 1 KB |
|  experimento_08.csv | 3/10/2026 23:32 | Microsoft Excel C... | 1 KB |
|  experimento_09.csv | 3/10/2026 23:33 | Microsoft Excel C... | 1 KB |
|  experimento_10.csv | 3/10/2026 23:33 | Microsoft Excel C... | 1 KB |

## 2.4 - Metodología de medición

Las mediciones se realizaron en la computadora cliente para mensajes de **10, 100, 1.000, 10.000, 100.000 y 1.000.000 bytes**. En cada repetición se estableció una nueva conexión TCP y se informó al servidor la cantidad de bytes que recibiría.

Se utilizó `CLOCK_MONOTONIC` para medir las duraciones, expresadas en nanosegundos, evitando que los ajustes de la hora del sistema alteraran los intervalos.

Cada ejecución realizó **10 repeticiones por tamaño** y guardó sus promedios en un archivo CSV.

Al ejecutar el programa completo diez veces, se obtuvieron **100 mediciones por tamaño**, agrupadas en diez promedios, a partir de los cuales calculamos la media general.

El **tiempo de escritura** representa la duración de la llamada local y no garantiza que los datos ya hayan llegado al servidor.

El **tiempo de lectura** incluye la espera de la respuesta, por lo que puede variar con el tamaño enviado aunque la confirmación siempre tenga cuatro bytes.

## 2.5 - Análisis de resultados

Se calcularon las **medias generales** de los tiempos de escritura y lectura a partir de las diez ejecuciones. También se obtuvo la **desviación estándar** entre los diez promedios para evaluar la variación de los resultados.

En esta tabla, todos los tiempos se expresan en **microsegundos ( $\mu$ s)**.

| Bytes enviados | <code>write()</code> - media ± desviación (μs) | <code>read()</code> - media ± desviación (μs) |
|----------------|------------------------------------------------|-----------------------------------------------|
| 10             | 4,50 ± 1,01                                    | 57.660,00 ± 1.020,00                          |
| 100            | 4,92 ± 1,04                                    | 58.550,00 ± 2.500,00                          |
| 1.000          | 5,28 ± 0,72                                    | 58.080,00 ± 1.780,00                          |
| 10.000         | 26,22 ± 14,66                                  | 11.190,00 ± 6.290,00                          |
| 100.000        | 76,24 ± 21,36                                  | 144.820,00 ± 27.810,00                        |
| 1.000.000      | 96.581,69 ± 39.954,88                          | 489.230,00 ± 98.960,00                        |

La desviación indicada corresponde a la dispersión **entre los promedios de las diez ejecuciones**, no entre las 100 mediciones individuales. Se utilizaron microsegundos para la escritura y milisegundos para la lectura por sus diferentes órdenes de magnitud.

### ¿Los tiempos son proporcionales a la cantidad de bytes?

Para analizar la proporcionalidad se calculó, por separado para `write()` y `read()`, el cociente entre los tiempos medios de tamaños consecutivos:

$$R = \frac{\bar{t}(10N)}{\bar{t}(N)}$$

Aquí,  $N$  es la cantidad de bytes enviados y  $\bar{t}(N)$  es el tiempo promedio correspondiente. **El cociente  $R$**  representa el factor por el que cambia el tiempo cuando la cantidad de bytes se multiplica por diez.

| Incremento de bytes | $R$ de <code>write()</code> | $R$ de <code>read()</code> |
|---------------------|-----------------------------|----------------------------|
| 10 → 100            | 1,09                        | 1,02                       |
| 100 → 1.000         | 1,07                        | 0,99                       |
| 1.000 → 10.000      | 4,97                        | 0,19                       |
| 10.000 → 100.000    | 2,91                        | 12,94                      |
| 100.000 → 1.000.000 | 1.266,75                    | 3,38                       |

En `write()`, los dos primeros cocientes muestran tiempos relativamente estables. Los siguientes aumentos tampoco siguen el factor diez y, en el último tramo, se observa un incremento de aproximadamente **1.266,75 veces**.

En `read()`, los tiempos también permanecen relativamente estables en los dos primeros tramos. Luego disminuyen, aumentan 12,94 veces y finalmente

aumentan 3,38 veces. Aunque uno de los cocientes se aproxima al factor diez, ese comportamiento no se mantiene en los demás tamaños.

Por lo tanto, **no se observa una proporcionalidad directa entre la cantidad de bytes enviados y los tiempos medios medidos**, ni un tiempo de lectura constante para todos los tamaños.

Si fuera así los cocientes deberían dar valores cercanos a 10 en todos los casos.

---

### Comportamiento de `write()`

El gráfico muestra un tramo casi horizontal entre 10 y 1.000 bytes, seguido de un crecimiento moderado y un salto pronunciado para 1.000.000 de bytes.

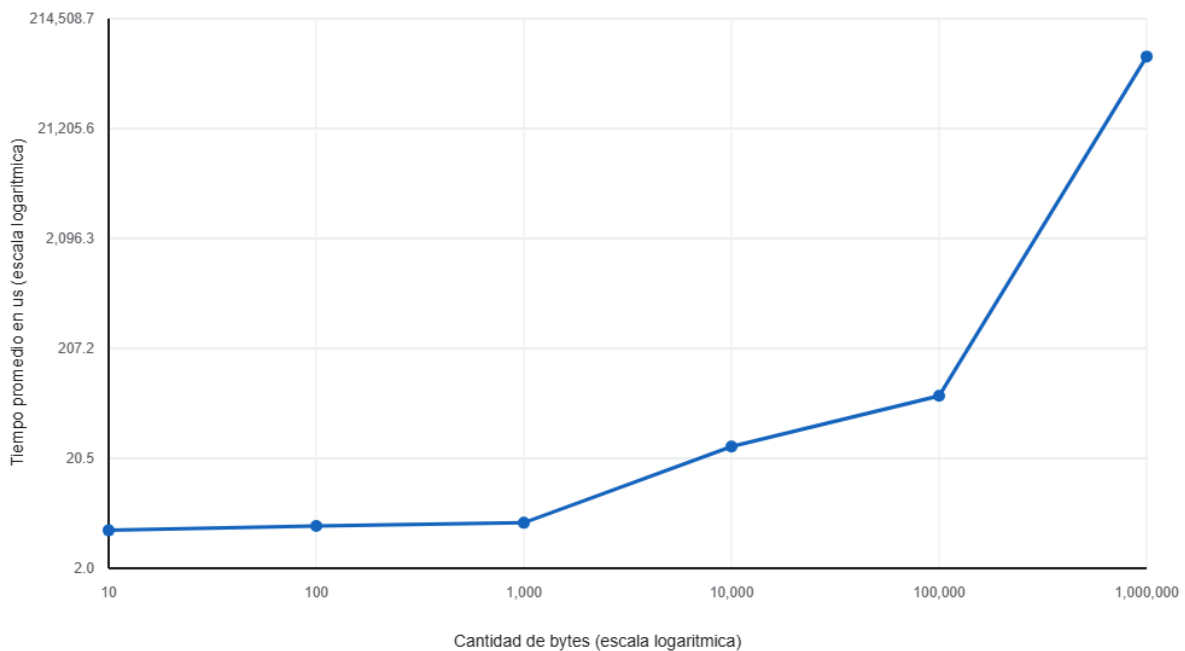
Como ambos ejes tienen escala logarítmica, se pueden distinguir estos cambios pese a las grandes diferencias entre los tiempos.

Para mensajes pequeños, una posible explicación es que predominen los costos de ejecutar la llamada y copiar los datos al buffer de envío.

Para el mensaje más grande, cuyo tiempo alcanza aproximadamente **96.582 microsegundos**, la escritura podría tener que esperar a que se libere espacio en ese buffer.

En un socket bloqueante, la falta de espacio puede demorar el envío. Esta explicación es compatible con el resultado, pero no fue comprobada mediante mediciones del estado de los buffers.

### write(): media de diez ejecuciones



### Comportamiento de `read()`

El gráfico presenta una forma irregular, los tres tamaños menores tienen tiempos similares, aparece una disminución para 10.000 bytes y luego los tiempos aumentan para los mensajes mayores.

Esto muestra que recibir siempre cuatro bytes no produce necesariamente una duración constante.

La diferencia entre los tiempos respecto al write es porque la lectura comienza antes de que esté disponible la confirmación y, por ello, incluye la espera hasta que el servidor complete la recepción, verifique los datos y responda.

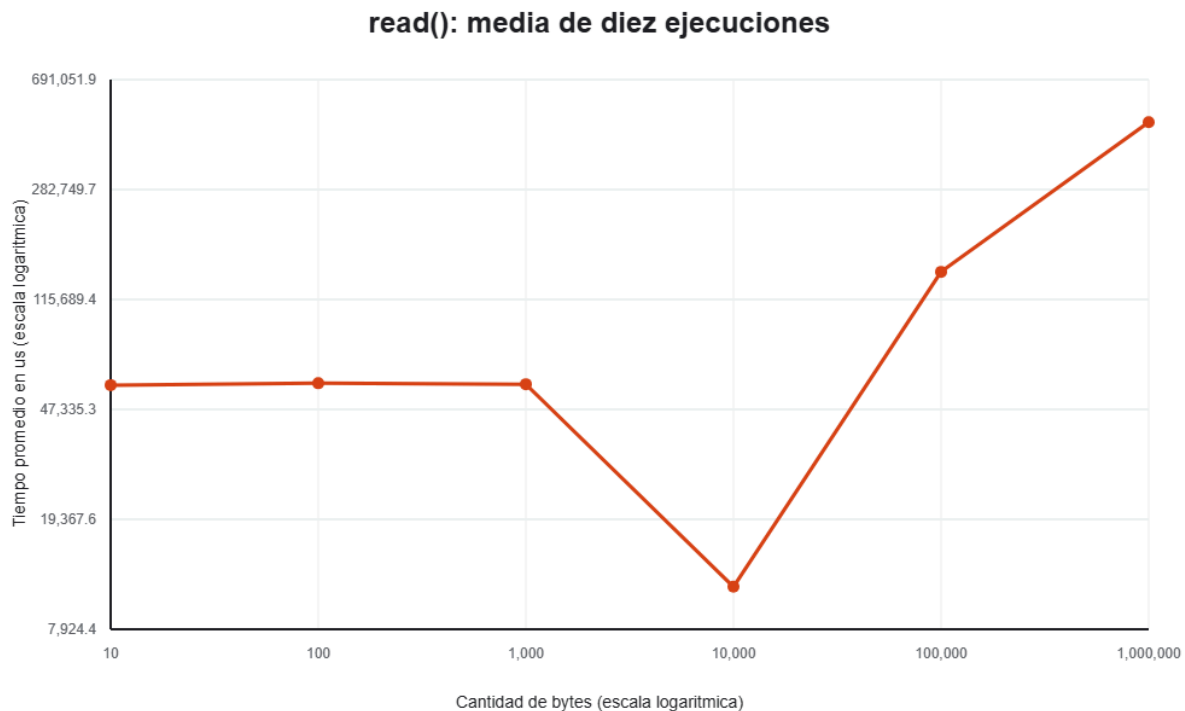
El incremento para mensajes grandes puede estar relacionado con esa mayor cantidad de trabajo y con la duración del intercambio.

La disminución para 10.000 bytes es un comportamiento extraño. No parece deberse a una medición aislada, ya que este comportamiento se observa en distintas ejecuciones y se mantiene en la media de las 100 iteraciones.

Una hipótesis es que TCP introduzca esperas al transmitir mensajes pequeños para agrupar datos o recibir confirmaciones. Con 10.000 bytes, habría suficientes datos para completar paquetes y enviarlos sin algunas de esas esperas, reduciendo el tiempo durante el cual el cliente queda bloqueado en

`read()`.

Para comprobarlo habría que observar el tráfico de red e identificar si esas esperas aparecen con los mensajes pequeños y disminuyen con los de 10.000 bytes. Lo cual está fuera del alcance del experimento.



---

### Variación entre ejecuciones

La desviación estándar permite complementar los gráficos de medias, que por sí solos no muestran cuánto cambiaron los resultados entre ejecuciones.

Para la lectura de 10.000 bytes representa aproximadamente el **56 % de la media**, mientras que para la escritura de 1.000.000 de bytes alcanza el **41 %**.

Estos valores indican una dispersión considerable y justifican evitar conclusiones basadas en una única ejecución.

De nuevo, lo más apropiado es realizar varias iteraciones del experimento.

Entre los factores que podrían contribuir a esa variación se encuentran las condiciones del enlace Wi-Fi, la planificación de los procesos, el estado de los buffers, además de WSL y el proxy TCP que también forman parte del recorrido de comunicación.

---

- 3 Desarrollar los mismos experimentos de comunicaciones que en el caso anterior, con las mismas computadoras, pero ahora con la misma cantidad de datos en un sentido y en el inverso (experimento "ping-pong"). El tiempo total "de ida y vuelta" de los datos dividido por 2 es el que se utiliza para una estimación del tiempo de mensajes en una dirección. Comparar los tiempos del ejercicio anterior con los que se obtienen en este experimento para cada cantidad de bytes.

### **3.1 - Computadoras y configuración de red**

Se utilizaron las mismas computadoras y conexiones descritas en el ejercicio 02. La computadora 1 ejecutó el servidor en Ubuntu mediante WSL 2 y la computadora 2 ejecutó el cliente en Debian mediante WSL.

Se mantuvieron los roles y la configuración de red para permitir la comparación entre ambos experimentos.

### **3.2 - Preparación del entorno de comunicación**

Se conservó el proxy TCP y la regla de firewall configurados anteriormente. El cliente utilizó como destino `192.168.100.6:5000`, y el proxy comunicó ese punto de entrada con `172.31.25.103:5000`, correspondiente a Ubuntu.

La preparación consistió en reemplazar los programas del ejercicio 02 por `server_pingpong` y `client_pingpong`.

|                                                                                     |                   |                 |               |       |
|-------------------------------------------------------------------------------------|-------------------|-----------------|---------------|-------|
|  | resultados        | 3/10/2026 23:42 | File folder   |       |
|  | client_pingpong   | 27/9/2026 15:35 | File          | 17 KB |
|  | client_pingpong.c | 27/9/2026 15:32 | C Source File | 8 KB  |
|  | server_pingpong   | 27/9/2026 15:35 | File          | 17 KB |
|  | server_pingpong.c | 27/9/2026 15:32 | C Source File | 6 KB  |

En este caso, el servidor devuelve los mismos datos que recibe, permitiendo medir el intercambio completo de ida y vuelta.

### **3.3 - Ejecución y almacenamiento de resultados**

En la computadora servidor, desde la terminal de Ubuntu y la carpeta del ejercicio 03, se ejecutó:

```
./server_pingpong 5000
```

El nuevo servidor se mantuvo en ejecución durante toda la serie de pruebas.

```
taiel@Taiel-PC: /mnt/e/@ FACULTAD/04 - cuarto año/segundo semestre/PDyTR/Prácticas/Práctica 2/03
taiel@Taiel-PC: /mnt/e/@ FACULTAD/04 - cuarto año/segundo semestre/PDyTR/Prácticas/Práctica 2/03$ ./server_pingpong 5000
Server listening on port 5000
```

Con el servidor disponible, el cliente puede ejecutarse desde Debian mediante:

```
./client_pingpong 192.168.100.6 5000
```

Para realizar diez ejecuciones completas y almacenar sus resultados en archivos CSV independientes, se utilizó:

```
mkdir -p resultados/serie_10

for i in $(seq -w 1 10); do
    echo "Experimento ping-pong $i de 10"
    ./client_pingpong 192.168.100.6 5000 \
        > "resultados/serie_10/experimento_${i}.csv" || break
    sleep 1
done
```

```
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03/resultados/serie_10
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03$ mkdir -p resultados/serie_10
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03$
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03$ for i in $(seq -w 1 10); do
> echo "Experimento ping-pong $i de 10"
> ./client_pingpong 192.168.100.6 5000 \
> > "resultados/serie_10/experimento_${i}.csv" || break
> sleep 1
> done
Experimento ping-pong 01 de 10
Experimento ping-pong 02 de 10
Experimento ping-pong 03 de 10
Experimento ping-pong 04 de 10
Experimento ping-pong 05 de 10
Experimento ping-pong 06 de 10
Experimento ping-pong 07 de 10
Experimento ping-pong 08 de 10
Experimento ping-pong 09 de 10
Experimento ping-pong 10 de 10
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03$ ls
client_pingpong client_pingpong.c resultados server_pingpong server_pingpong.c
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03$ cd resultados/
marce@DESKTOP-3UBP3KG: /mnt/c/Users/mimat/Desktop/.borrar/03/resultados$ ls
experimento_01.csv experimento_02.csv experimento_03.csv experimento_04.csv experimento_05.csv experimento_06.csv experimento_07.csv experimento_08.csv experimento_09.csv experimento_10.csv
```

De la misma forma que en el experimento previo, Cada ejecución contiene **10 repeticiones por tamaño**, por lo que la serie completa reúne **100 mediciones de ida y vuelta por tamaño**, agrupadas en diez promedios.

Los CSV registran el tamaño del mensaje, la cantidad de repeticiones, el tiempo promedio de ida y vuelta y su mitad como estimación del tiempo en una dirección.

Los archivos se conservaron para calcular las medias generales, elaborar las tablas y los gráficos y comparar los resultados con los del ejercicio 02.

|                                                                                     |                    |                 |                      |      |
|-------------------------------------------------------------------------------------|--------------------|-----------------|----------------------|------|
|  | experimento_01.csv | 3/10/2026 23:42 | Microsoft Excel C... | 1 KB |
|  | experimento_02.csv | 3/10/2026 23:43 | Microsoft Excel C... | 1 KB |
|  | experimento_03.csv | 3/10/2026 23:43 | Microsoft Excel C... | 1 KB |
|  | experimento_04.csv | 3/10/2026 23:43 | Microsoft Excel C... | 1 KB |
|  | experimento_05.csv | 3/10/2026 23:44 | Microsoft Excel C... | 1 KB |
|  | experimento_06.csv | 3/10/2026 23:44 | Microsoft Excel C... | 1 KB |
|  | experimento_07.csv | 3/10/2026 23:44 | Microsoft Excel C... | 1 KB |
|  | experimento_08.csv | 3/10/2026 23:45 | Microsoft Excel C... | 1 KB |
|  | experimento_09.csv | 3/10/2026 23:45 | Microsoft Excel C... | 1 KB |
|  | experimento_10.csv | 3/10/2026 23:46 | Microsoft Excel C... | 1 KB |

### 3.4 - Metodología de medición

Los calculos y mediciones se realizan de forma muy similar a la del experimento previo.

Se utilizó un intercambio **ping-pong**: el cliente envía un mensaje de  $N$  bytes y el servidor devuelve esos mismos  $N$  bytes. Se probaron tamaños de **10, 100, 1.000, 10.000, 100.000 y 1.000.000 bytes**, manteniendo la misma cantidad de datos en ambos sentidos.

En cada repetición, el cliente establece una conexión TCP e informa al servidor el tamaño del mensaje. Luego registra el instante anterior al envío y el instante posterior a la recepción completa de la respuesta:

$$t_{\text{ida y vuelta}} = t_{\text{fin}} - t_{\text{inicio}}$$

La duración se obtiene mediante `CLOCK_MONOTONIC`.

Las funciones de envío y recepción repiten las operaciones necesarias hasta completar los  $N$  bytes, evitando considerar terminado un intercambio parcial. Además, el cliente verifica que los datos devueltos coincidan con los enviados después de detener la medición.

Siguiendo el enunciado, se calcula:

$$t_{\text{estimado en una dirección}} = \frac{t_{\text{ida y vuelta}}}{2}$$

Este valor es una **estimación**, ya que supone tiempos similares en ambos sentidos e incluye las esperas y el procesamiento realizados durante el intercambio, como la verificación de datos del servidor.

### **3.5 - Análisis de resultados y comparación con el ejercicio 02**

Se calcularon las medias generales a partir de las diez ejecuciones del experimento ping-pong. Todos los tiempos están en **microsegundos ( $\mu\text{s}$ )**.

#### **Resultados del intercambio ping-pong**

| Bytes en cada sentido | Ida y vuelta<br>media $\pm$ desviación ( $\mu\text{s}$ ) | Estimación en una dirección<br>media $\pm$ desviación ( $\mu\text{s}$ ) |
|-----------------------|----------------------------------------------------------|-------------------------------------------------------------------------|
| 10                    | 60.032,82 $\pm$ 3.631,78                                 | 30.016,41 $\pm$ 1.815,89                                                |
| 100                   | 60.893,23 $\pm$ 5.230,80                                 | 30.446,62 $\pm$ 2.615,40                                                |
| 1.000                 | 62.388,42 $\pm$ 4.400,88                                 | 31.194,21 $\pm$ 2.200,44                                                |
| 10.000                | 21.353,59 $\pm$ 9.297,71                                 | 10.676,80 $\pm$ 4.648,85                                                |
| 100.000               | 265.766,63 $\pm$ 97.477,53                               | 132.883,31 $\pm$ 48.738,77                                              |
| 1.000.000             | 1.513.168,46 $\pm$ 303.269,72                            | 756.584,23 $\pm$ 151.634,86                                             |

Tanto la media como la desviación de la estimación en una dirección se obtienen dividiendo por dos sus valores de ida y vuelta.

### ¿Los tiempos son proporcionales a la cantidad de bytes?

Se calculó el cociente entre los tiempos medios de los tamaños que son consecutivos:

$$R = \frac{\bar{t} \text{ ida y vuelta}(10N)}{\bar{t} \text{ ida y vuelta}(N)}$$

También es válido para la estimación en una dirección, porque dividir ambos tiempos por dos no modifica su relación.

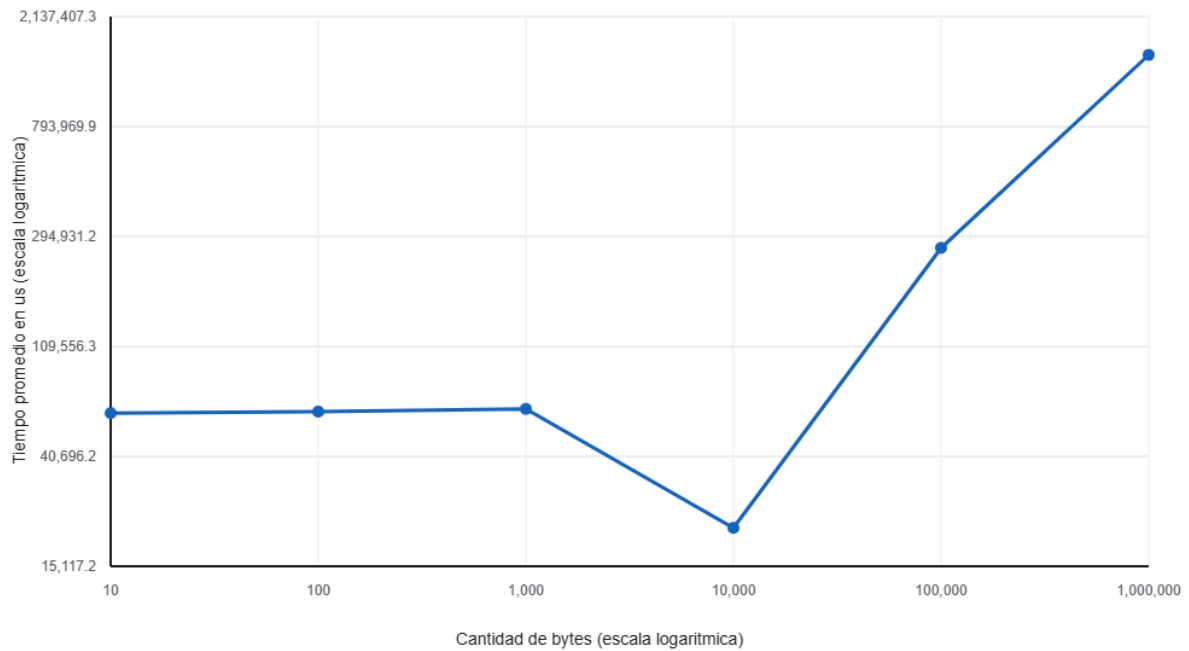
| Tamaño anterior → siguiente (bytes) | Cociente $\backslash(R\backslash)$ |
|-------------------------------------|------------------------------------|
| 10 → 100                            | 1,01                               |
| 100 → 1.000                         | 1,02                               |
| 1.000 → 10.000                      | 0,34                               |
| 10.000 → 100.000                    | 12,45                              |
| 100.000 → 1.000.000                 | 5,69                               |

Los cocientes no se mantienen próximos a 10, por lo que **no se observa una proporcionalidad directa** entre la cantidad de bytes y el tiempo del intercambio.

### Comportamiento de Ida y Vuelta

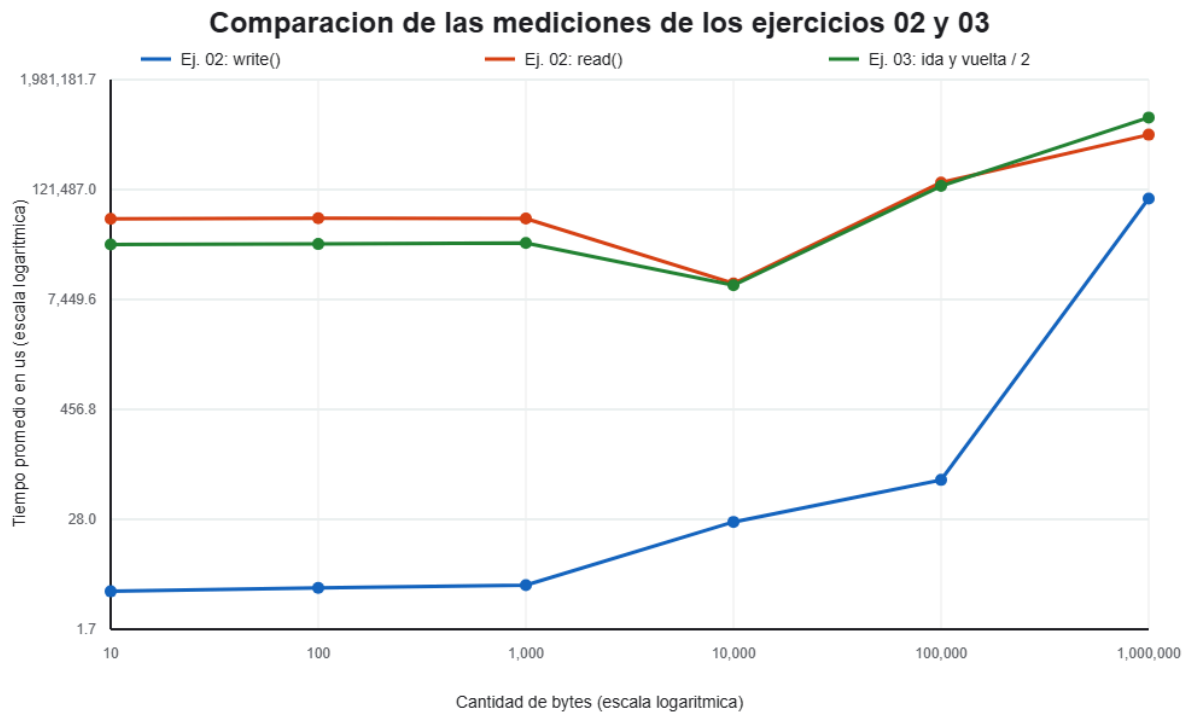
El gráfico presenta un tramo inicial estable, una disminución para 10.000 bytes, de la misma forma que en el ejercicio 02, y un crecimiento pronunciado para los tamaños mayores.

### Ping-pong: tiempo medio de ida y vuelta



### Comparación con el ejercicio 02

| Bytes enviados | Ej. 02:<br>media de <code>write()</code><br>(μs) | Ej. 02:<br>media de <code>read()</code><br>(μs) | Ej. 03:<br>media de ida y vuelta /<br>2 (μs) |
|----------------|--------------------------------------------------|-------------------------------------------------|----------------------------------------------|
| 10             | 4,50                                             | 57.657,30                                       | 30.016,41                                    |
| 100            | 4,92                                             | 58.548,15                                       | 30.446,62                                    |
| 1.000          | 5,28                                             | 58.082,92                                       | 31.194,21                                    |
| 10.000         | 26,22                                            | 11.193,55                                       | 10.676,80                                    |
| 100.000        | 76,24                                            | 144.815,44                                      | 132.883,31                                   |
| 1.000.000      | 96.581,69                                        | 489.227,30                                      | 756.584,23                                   |



El gráfico muestra que el tiempo de `write()` es considerablemente menor que la estimación ping-pong. Esto se explica por el alcance de las mediciones, `write()` puede finalizar cuando los datos quedan en los buffers locales, mientras que el ping-pong espera hasta recibir completamente el mensaje devuelto.

Para los tres tamaños menores, la estimación del ejercicio 03 es aproximadamente la mitad del tiempo de lectura del ejercicio 02. Esto NO demuestra una comunicación más rápida... La estimación `ping-pong` divide por dos un intercambio completo, mientras que el `read()` incluye la espera de la confirmación sin aplicar esa división.

Para 1.000.000 de bytes, la estimación ping-pong supera el tiempo de lectura del ejercicio 02. Una diferencia relevante es que el servidor ahora devuelve **1.000.000 de bytes**, frente a los **cuatro bytes de confirmación** del experimento anterior.

### Variación entre ejecuciones

La desviación estándar para 10, 100 y 1000 representa aproximadamente entre el **6% y el 9% de la media**. Para 10.000, 100.000 y 1.000.000 de bytes alcanza aproximadamente el **44%, 37% y 20%**, respectivamente.

Estos valores muestran que la dispersión es mayor para algunos tamaños y justifican acompañar las medias con una medida de variación.

Al igual que en el ejercicio 02, la estimación obtenida incluye el tiempo de comunicación a través de Wi-Fi, Ethernet, WSL y el proxy TCP; por ello, no representa exclusivamente el tiempo de transmisión de la red física.

- 4** Teniendo en cuenta los experimentos de tiempos realizados, desarrollar scripts para desplegar un ambiente de experimentación de comunicación en una computadora con Vagrant para los siguientes escenarios:
- Dos máquinas virtuales, cada una con un proceso de comunicaciones.
  - Una máquinas virtual con uno de los procesos de comunicaciones y el otro proceso de comunicaciones en el host.

En todos los casos deberían quedar los resultados disponibles para su posterior análisis. Se debe resolver el problema de "asincronismo" que genera un error si el proceso "cliente" inicia la ejecución antes que el proceso "servidor".

Consultar con un asistente con IA cuál es sería el mejor método experimental para estimar el tiempo de comunicaciones entre dos computadoras diferentes y comente si le parece correcto o puede identificar algún problema en la respuesta del asistente con IA.

#### 4.1 - Preparación del ambiente de experimentación

Para realizar los experimentos en una sola computadora, se preparó un entorno con **Vagrant y VirtualBox**, utilizando los archivos de la carpeta 04. El escenario B también requiere **Python 3** para ejecutar el servidor directamente en el host, que puede utilizar **Windows o Linux**.

El **Vagrantfile** configura VirtualBox como proveedor predeterminado y crea VM con **Ubuntu 24.04, un procesador virtual y 1.024 MB de RAM** cada una.

También instala las herramientas de compilación y **netcat**.

| Proceso  | Escenario A       | Escenario B                        |
|----------|-------------------|------------------------------------|
| Servidor | VM: 192.168.56.20 | Host Windows o Linux: 192.168.56.1 |
| Cliente  | VM: 192.168.56.21 | VM: 192.168.56.21                  |

Las direcciones de las VM están definidas en el `Vagrantfile`. La dirección del host corresponde al adaptador *host-only* de VirtualBox y debe verificarse antes de ejecutar el escenario B. Si es diferente, se actualiza en los comandos del servidor y del cliente. El firewall del host debe permitir la comunicación TCP por el puerto **5000**.

Se utilizan tres scripts:

- `script_servidor.sh` : compila y ejecuta el servidor C en la VM.
- `script_cliente.sh` : compila el cliente, espera a que el servidor esté disponible y realiza diez ejecuciones por defecto.
- `host_server.py` : ejecuta el servidor Python en el host para el escenario B.

Los scripts permiten realizar los experimentos `pingpong` y `timing`. La espera inicial evita iniciar las mediciones antes de que el servidor esté disponible y queda fuera de los tiempos medidos.

Los resultados se guardan en una carpeta nueva dentro de `04/experiments/resultados/`, accesible desde el host para su posterior análisis.

Al comparar ambos escenarios, debe considerarse que el servidor de A está implementado en C y el de B en Python. Por lo tanto, las diferencias de tiempo también pueden incluir diferencias en el procesamiento de los servidores. El procedimiento puede reproducirse en distintas computadoras, aunque los tiempos dependen del hardware, del sistema operativo y de la carga del equipo.

---

## 4.2 - Escenario A: comunicación entre dos VM

### 1. Iniciar las máquinas virtuales

Desde PowerShell, ubicado en la carpeta `04`, se selecciona el escenario A y se inicia el entorno:

```
$env:ESCENARIO = "A"
vagrant up
```



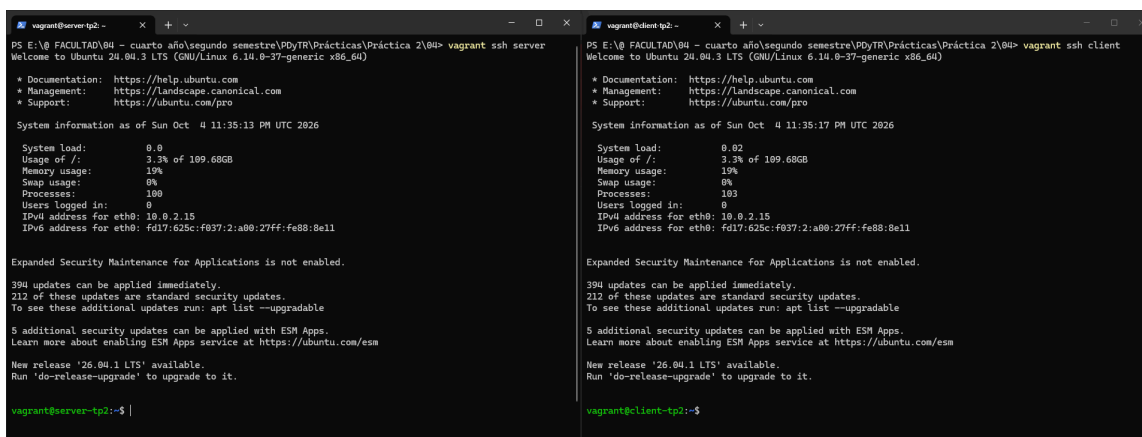
## 2. Abrir una terminal para cada VM

En una ventana de PowerShell se accede a la VM servidor:

```
vagrant ssh server
```

En otra ventana, también ubicada en 04 , se accede a la VM cliente:

```
vagrant ssh client
```



The image shows two terminal windows side-by-side. Both windows are titled 'vagrant@server-tp2' and 'vagrant@client-tp2'. They both display the Ubuntu 24.04.3 LTS login screen, including the Ubuntu logo, the text 'Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86\_64)', and links for documentation, management, and support. Below the login screen, both windows display system information as of Sun Oct 4 11:35:13 PM UTC 2026. The system information includes system load, usage of /, memory usage, swap usage, processes, users logged in, IPv4 address for eth0, and IPv6 address for eth0. Both windows also display a message about Expanded Security Maintenance for Applications (ESM) and a note about 394 updates being available immediately. The terminal windows are running on a Windows PowerShell environment, as indicated by the path 'PS E:\0 FACULTAD\04 - cuarto año\segundo semestre\PDYTR\Prácticas\Práctica 2\04'.

## 3. Ejecutar Scripts

El script genera **diez archivos CSV**, cada uno con los promedios de diez repeticiones por tamaño: en total, **100 mediciones por tamaño**. Estos archivos quedan disponibles dentro de 04/experiments/resultados/ .

Aplicamos el script para obtener resultados ambos experimentos *pingpong* y *timing*.

- Experimento ping-pong

1. Dentro de la VM servidor se inicia el experimento *ping-pong*:

```
bash /vagrant/script_servidor.sh pingpong
```

2. Se mantiene esa terminal abierta y, dentro de la VM cliente, se ejecuta:

```
bash /vagrant/script_cliente.sh pingpong 192.168.5  
6.20
```



Si la VM servidor del escenario A continúa encendida, primero se detiene su proceso con `Ctrl+C` y se apaga la VM.

En Windows:

```
$env:ESCENARIO = "A"
vagrant halt server
```

En Linux:

```
ESCENARIO=A vagrant halt server
```

Luego, desde la carpeta `04`, se selecciona el escenario B y **se inicia únicamente la VM cliente**.

## 1. Iniciar la VM cliente y verificar la dirección del host

- **En Windows, desde PowerShell:**

```
$env:ESCENARIO = "B"
vagrant up client
ipconfig
```

- **En Linux, desde una terminal:**

```
export ESCENARIO=B
vagrant up client
ip addr
```

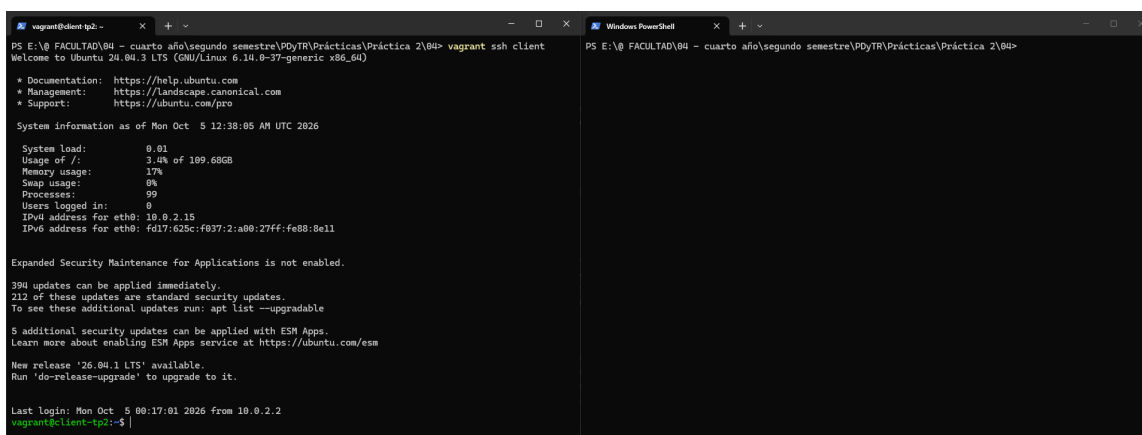
Se utiliza la dirección del adaptador *host-only* de VirtualBox. En este experimento se utiliza `192.168.56.1`; si el adaptador tiene otra dirección, debe reemplazarse en los comandos del servidor y del cliente.

## 2. Abrir las terminales del cliente y del servidor

En la misma terminal donde se seleccionó el escenario B, se accede a la VM cliente:

```
vagrant ssh client
```

Se abre una segunda terminal **en el host**, ubicada en la carpeta **04**, para ejecutar el servidor Python.



### 3. Ejecutar los scripts

Se realizan ambos experimentos, *pingpong* y *timing*.

- **Experimento ping-pong**

1. En la terminal **host**, se ejecuta el script para **iniciar el Servidor**, indicando el experimento **pingpong**.

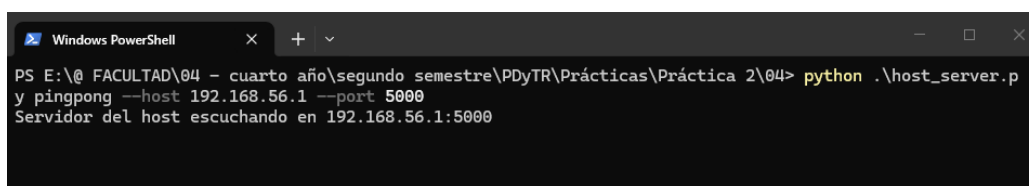
en **Windows**:

```
python .\host_server.py pingpong --host 192.168.56.1 --port 5000
```

en **Linux**:

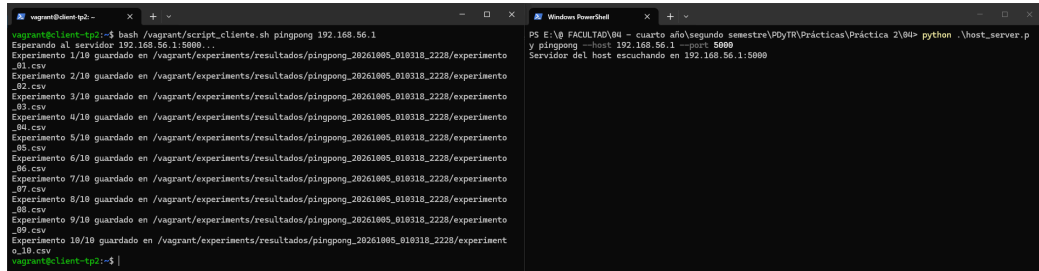
```
python3 ./host_server.py pingpong --host 192.168.56.1 --port 5000
```

El servidor queda escuchando y su terminal se mantiene abierta.



2. Dentro de la **VM cliente**, también indicando el experimento **pingpong**:

```
bash /vagrant/script_cliente.sh pingpong 192.168.56.1
```



- Experimento de tiempos de write() y read()

Cuando termina el cliente, se detiene el servidor con **Ctrl+C** y se inicia el experimento *timing*.

1. En la terminal **host**, se ejecuta el script para iniciar el Servidor pero **esta vez indicamos el experimento timing**

En el **host Windows**:

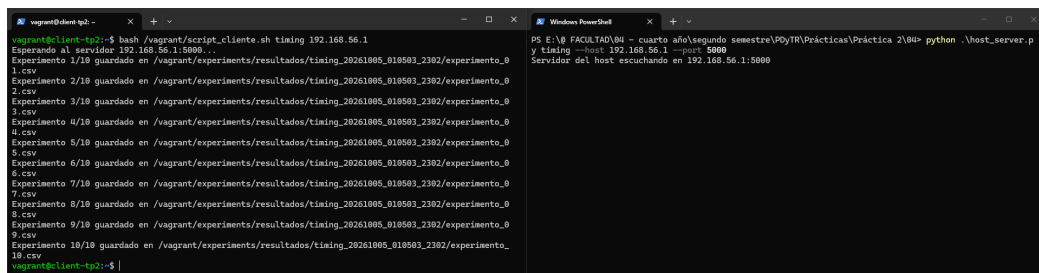
```
python .\host_server.py timing --host 192.168.56.1  
--port 5000
```

En el **host Linux**:

```
python3 ./host_server.py timing --host 192.168.56.1  
--port 5000
```

2. Dentro de la **VM cliente**, también indicando el experimento **timing**:

```
bash /vagrant/script_cliente.sh timing 192.168.56.1
```



#### 4.4 - ¿Cómo se resuelve el problema del Asincronismo?

El problema aparece cuando el cliente intenta conectarse antes de que el servidor esté escuchando. Para evitarlo, `script_cliente.sh` comprueba la disponibilidad del puerto mediante `nc` antes de ejecutar el programa cliente:

```
INTENTOS=0
until nc -z -w 1 "$SERVIDOR" "$PUERTO" >/dev/null 2>&1; do
    INTENTOS=$((INTENTOS + 1))
    if (( INTENTOS >= 60 )); then
        echo "El servidor no esta disponible" >&2
        exit 1
    fi
    sleep 1
done
```

Si la conexión falla, el script espera **un segundo** y vuelve a intentarlo. Cuando el servidor está disponible, sale del bucle e inicia el experimento. Si se alcanzan **60 intentos fallidos**, finaliza con un mensaje de error.

Así, el cliente puede iniciarse primero y esperar al servidor en ambos escenarios. **Esta espera queda fuera de los tiempos medidos**, porque ocurre antes de ejecutar el programa C que realiza las mediciones.

---

#### 4.5 - Análisis y consulta con IA - DeepSeek

Estoy realizando una práctica de comunicaciones entre dos computadoras físicamente diferentes, conectadas por una red y utilizando TCP. Quiero estimar el tiempo de comunicación para mensajes de 10, 100, 1.000, 10.000, 100.000 y 1.000.000 de bytes.

¿Cuál consideras que sería el mejor método experimental para estimar el tiempo que tarda un mensaje en viajar de una computadora a la otra?



## Mejor método experimental para estimar tiempos de comunicación TCP

Para estimar el tiempo de comunicación entre dos computadoras con TCP, el método más riguroso es un **experimento de ida y vuelta (RTT)** con múltiples repeticiones y separación de componentes. Te detallo el enfoque:

### 1. Diseño básico: cliente-servidor con eco

- **Servidor:** escucha en un puerto TCP, recibe N bytes y los devuelve inmediatamente (eco).
- **Cliente:** envía el mensaje, mide el tiempo hasta recibir la respuesta completa, y calcula el RTT.
- Repetir para cada tamaño: 10, 100, 1 KB, 10 KB, 100 KB, 1 MB.

Esto te da el **tiempo de ida y vuelta**, no el de un solo sentido. Para estimar el unidireccional, asumiendo simetría:  $t_{ida} \approx RTT / 2$ .

## 2. Consideraciones críticas para que sea válido

### a) Calentamiento (warm-up)

Las primeras conexiones incluyen handshake TCP, resolución ARP, posible DNS, y arranque en frío de la pila de red. Descartá las primeras iteraciones (p. ej. las 10 primeras) y medí solo el régimen estable.

### b) Múltiples repeticiones

No midas una sola vez. Hacé al menos 30–100 repeticiones por tamaño y reportá **mediana**, **media**, **desvío estándar** y **percentiles (p95, p99)**. Los promedios solos esconden la cola.

### c) Aislar el costo de establecimiento de conexión

Si medís `connect() + send() + recv()` incluí el handshake TCP (3 vías) en cada medición. Para medir solo la transferencia:

- Reutilizá la misma conexión para todas las repeticiones de un tamaño, o
- Medí por separado `tiempo_conexion` y `tiempo_transferencia`.

### d) Evitar el efecto de Nagle y delayed ACK

Deshabilitá Nagle con `TCP_NODELAY` en ambos extremos. De lo contrario, mensajes pequeños pueden quedar bufferizados y distorsionar la medición de los tamaños chicos.

### e) Sincronización de relojes

Si querés medir tiempo unidireccional real (no asumir simetría), necesitás sincronizar relojes con PTP (mejor) o NTP (suficiente para ms). Aun así, el RTT evita este problema y suele ser preferible.

## 3. Metodología recomendada (paso a paso)

1. **Fijar condiciones:** misma red, mismos equipos, sin otras cargas, cable en lugar de WiFi si es posible.
2. **Handshake inicial:** abrir la conexión una vez, hacer warm-up.
3. **Para cada tamaño S:**
  - Repetir R veces (ej. 100):
    - `t0 = now()`
    - `send(S bytes)`
    - `recv(S bytes)` (bloqueante hasta completar)
    - `t1 = now()`
    - `RTT_i = t1 - t0`
  - Guardar todos los `RTT_i`.
4. **Calcular estadísticos** por tamaño: mediana, media,  $\sigma$ , p95, p99, mínimo.
5. **Repetir el experimento en varios momentos del día** para ver variabilidad.

## 4. Análisis esperado

El tiempo total se modela como:

text

Copy

Download

$$t(S) \approx t_{\text{fijo}} + S / \text{BW}_{\text{efectivo}} + \text{latencia}$$

- **tamaños chicos (10–100 B):** domina el overhead fijo (syscalls, stack, latencia).
- **tamaños grandes ( $\geq 100$  KB):** domina el ancho de banda efectivo.
- Gráfica  $t$  vs  $S$  y vas a ver una recta cuya pendiente es  $1/\text{BW}$  y cuya ordenada al origen es el overhead fijo.

**Ojo con 1 MB:** puede superar el MSS y requerir múltiples segmentos; además puede llenar buffers del socket y provocar bloqueos. Verificá que el `recv` del servidor y el `send` del cliente manejen correctamente transferencias parciales (no asumas que un `send` envía todo ni que un `recv` recibe todo).

Se consultó al asistente **DeepSeek** sobre el mejor método experimental para estimar el tiempo de comunicación TCP entre dos computadoras diferentes.

Su propuesta fue realizar un experimento **ping-pong**, enviando y devolviendo la misma cantidad de bytes, repetir las mediciones y estimar el tiempo en una dirección mediante:

$$t_{\text{estimado}} = \frac{RTT}{2}$$

La propuesta resulta adecuada para esta práctica, y de hecho, se corresponde con uno de los experimentos realizados en la misma, donde el establecimiento de la conexión queda fuera del tiempo medido.

Sin embargo, se identificaron 2 limitaciones:

- **RTT/2 es una estimación:** esta suponiendo tiempos similares en ambos sentidos e incluye procesamiento del servidor y esperas del sistema operativo. Por eso, no representa exclusivamente el tiempo de transmisión de la red.
- **No puede asegurarse una relación lineal entre tamaño y tiempo:** los buffers, la congestión y el comportamiento de TCP pueden producir variaciones y cambios de tendencia.

Por lo tanto, se considera una recomendación práctica y razonable, pero la afirmación de que es el método "más riguroso" (que se ve en la primera

captura) es dudoso. La utilidad de este método depende de qué tiempo se pretende medir y de la precisión necesaria.

- 5** Explique si los resultados de tiempo de comunicaciones del experimento del ej. anterior se ven afectados por el orden de ejecución y las diferencias de tiempo de ejecución iniciales de los programas utilizados ¿qué sucede si por alguna razón hay una diferencia de 10 segundos en el inicio de la ejecución entre un programa y otro? Sugerencia: incluya un “sleep” de 10 segundos entre la ejecución de diferentes partes de los programas y comente.

Para comprobar el efecto de una diferencia de **10 segundos** entre el inicio de los procesos, se utiliza el experimento *ping-pong* del escenario A, con ambas VM encendidas y sus terminales abiertas.

Se compararon dos condiciones en el escenario A: el servidor iniciado antes que el cliente y el servidor iniciado después de una pausa de diez segundos, cuando el cliente ya estaba esperando.

El tiempo total se midió dentro de la VM cliente, incluyendo compilación, espera inicial, ejecuciones y pausas del script:

| Condición                              | Tiempo total del script (s) |
|----------------------------------------|-----------------------------|
| Servidor iniciado primero              | 18,66                       |
| Cliente primero y servidor con retraso | 27,73                       |

La pausa registrada duró **10,011 segundos** y el tiempo total aumentó **9,07 segundos**. La diferencia no coincide exactamente con la pausa, porque también varía el tiempo empleado por las demás operaciones.

Los tiempos de ida y vuelta guardados en los CSV fueron:

| Bytes  | RTT sin retraso: media ± desviación (ms) | RTT con retraso: media ± desviación (ms) |
|--------|------------------------------------------|------------------------------------------|
| 10     | 2,55 ± 0,57                              | 2,57 ± 0,70                              |
| 100    | 2,38 ± 0,62                              | 2,27 ± 0,47                              |
| 1.000  | 2,24 ± 0,39                              | 2,22 ± 0,29                              |
| 10.000 | 5,64 ± 6,60                              | 3,61 ± 0,50                              |

| Bytes     | RTT sin retraso: media ± desviación (ms) | RTT con retraso: media ± desviación (ms) |
|-----------|------------------------------------------|------------------------------------------|
| 100.000   | 10,53 ± 3,42                             | 11,35 ± 3,30                             |
| 1.000.000 | 30,53 ± 4,07                             | 33,08 ± 5,52                             |

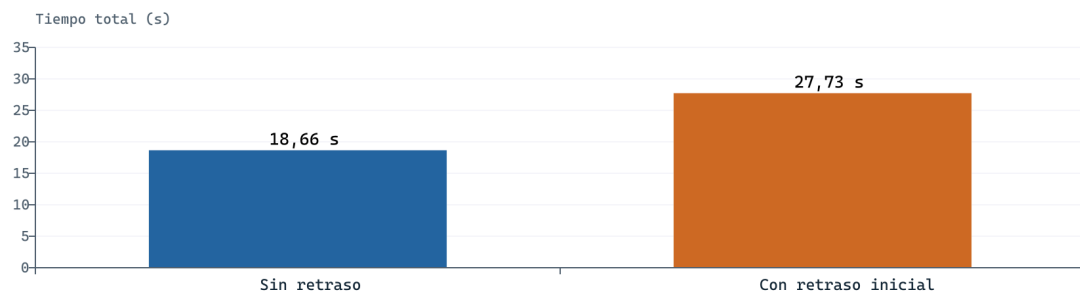
Para 10.000 bytes, una ejecución sin retraso presentó un promedio de **24,35 ms**, elevando la media y la dispersión de esa condición. Los datos no permiten determinar la causa exacta de esta variación.

### Efecto de un retraso de 10 segundos al iniciar el servidor

Escenario A: dos VM Ubuntu en VirtualBox · cliente 192.168.56.21 · servidor 192.168.56.20:5000

#### A. Tiempo total del script cliente

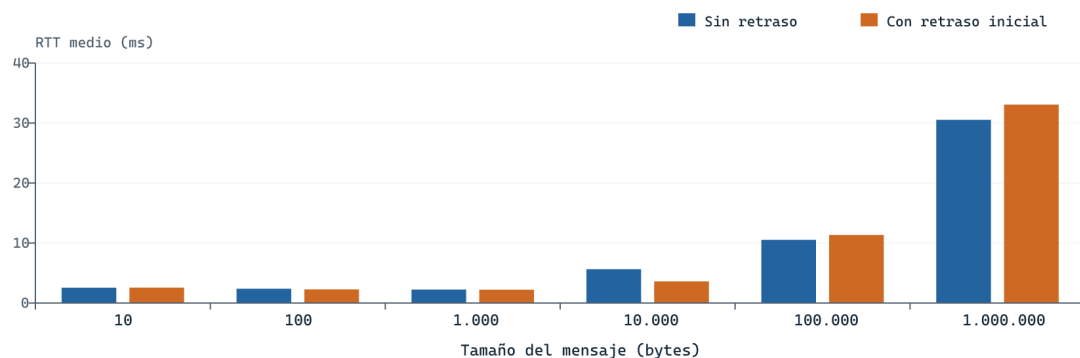
Incluye compilación, espera inicial, diez ejecuciones y pausas entre ejecuciones.



Pausa registrada: 10,011 s. Aumento del tiempo total: 9,07 s.

#### B. Tiempo de ida y vuelta de los mensajes (RTT)

Media de 100 mediciones por tamaño y condición; la espera inicial ocurre antes de cronometrar.



Los RTT se mantienen en milisegundos: no se suman diez segundos a cada medición.

Fuente: 20 CSV de ej05\_asincronismo\_d50108a7. Un tiempo total del script por condición.

- El panel superior muestra el aumento de la duración total del script.
- En el inferior, los RTT permanecen en el orden de los **milisegundos**, sin un aumento de diez segundos por mensaje-

### Conclusión:

Los resultados son compatibles con la ubicación de los cronómetro. **La espera inicial retrasa la finalización del experimento, pero queda fuera del intervalo utilizado para medir el RTT.**

Las diferencias entre los promedios no demuestran por sí solas un efecto del orden de inicio sobre la comunicación.

---